Dominio 4: prompt engineering e output strutturato (20%)

Questo dominio tratta la definizione di criteri espliciti, il few-shot prompting, la generazione garantita di output strutturati tramite `tool_use` e schemi JSON, i cicli di convalida e nuovo tentativo, l'elaborazione batch e l'architettura di revisione in più passaggi.

4.1 Criteri espliciti

Il problema delle istruzioni vaghe

Le istruzioni vaghe producono risultati incoerenti perché il modello le interpreta in modo diverso ogni volta.

Errato (vago):

```
"Segnala i problemi con prudenza."
"Riporta solo risultati molto affidabili."
"Usa il tuo miglior giudizio per valutarne la gravità."
```

Corretto (esplicito):

```
"Segnala un commento come obsoleto SOLO quando il comportamento descritto
contraddice direttamente quello effettivo del codice. NON segnalare commenti
semplicemente incompleti o che potrebbero essere più dettagliati."
```

Struttura dei criteri espliciti

Per ogni tipo di valutazione che il modello deve effettuare, definisci:

1. **Cosa rientra nei criteri** — Condizioni specifiche che attivano la valutazione
2. **Cosa ne resta escluso** — Casi limite che NON dovrebbero attivarla
3. **Esempi** — Coppie concrete di input/output che mostrano il confine

```
Segnala un PROBLEMA DI SICUREZZA quando:
  - l'input dell'utente viene passato a query SQL senza parametrizzazione;
  - l'input dell'utente viene renderizzato in HTML senza escaping;
  - le credenziali sono inserite direttamente nei file sorgente.

NON segnalare:
  - query ORM (sono già parametrizzate);
  - il rendering dell'input utente in template lato server con escaping
automatico;
  - credenziali caricate da variabili d'ambiente.
```

Impatto dei falsi positivi

Un alto tasso di falsi positivi in una categoria compromette la fiducia in TUTTE le categorie, anche in quelle accurate. Gli sviluppatori iniziano a ignorare ogni avviso.

Soluzione: se una categoria genera molti falsi positivi, disattivala temporaneamente e migliora i prompt dei criteri prima di riattivarla. Non lasciare che una categoria rumorosa riduca il valore di quelle accurate.

4.2 Few-shot prompting

Quando usare il few-shot

- Scenari ambigui in cui le istruzioni in prosa non sono abbastanza precise.
- Attività in cui è necessario mostrare il formato o lo schema di ragionamento.
- Riduzione delle allucinazioni nelle attività di estrazione.
- Dimostrazione della gestione dei casi limite.

Quanti esempi usare

Da 2 a 4 esempi mirati è il numero ideale. Troppi pochi potrebbero non coprire lo schema; troppi sprecano contesto e possono causare un adattamento eccessivo agli esempi.

Cosa includere negli esempi efficaci

Ogni esempio dovrebbe mostrare:

1. **Input** — Cosa riceve il modello
2. **Output** — Cosa dovrebbe produrre
3. **Ragionamento** — PERCHÉ l'output è corretto (non solo cosa fare, ma anche perché)

```
Esempio 1:
Input: "// Restituisce il nome completo dell'utente"
Code: function getName(user) { return user.firstName; }
Output: {
  "issue": "Il commento dichiara il nome completo, ma la funzione restituisce solo
firstName",
  "severity": "medium",
```

```
  "location": "riga 1",
  "reasoning": "Il commento dice 'nome completo', ma il codice restituisce solo
    firstName, non firstName + lastName. È una contraddizione fattuale."
}

Esempio 2:
Input: "// Convalida l'input"
Code: function validate(input) { return input.length > 0 && input.length < 100; }
Output: null
Ragionamento: "Il commento dice 'convalida l'input' e la funzione effettivamente
  convalida l'input. Il commento è vago ma non errato: non dichiara regole
  di convalida specifiche che contraddicano l'implementazione."
```

Il few-shot favorisce la generalizzazione

Gli esempi efficaci insegnano lo schema, non solo i casi specifici. Comprendendo il ragionamento alla base di ogni esempio, il modello dovrebbe generalizzare a situazioni nuove.

4.3 Output strutturato tramite tool_use

Il problema

Quando vuoi che Claude restituisca dati strutturati (JSON), la generazione di testo libero può produrre:

- JSON malformato;
- campi obbligatori mancanti;
- nomi di campo imprevisti;
- tipi di dati errati.

Soluzione 1: tool_use con schema

Definisci uno strumento che rappresenti la struttura di output desiderata:

```
{
  "name": "extract_invoice_data",
  "description": "Estrai dati strutturati da una fattura",
  "input_schema": {
    "type": "object",
    "properties": {
      "vendor_name": {
        "type": "string",
        "description": "L'azienda o la persona che ha emesso la fattura"
      },
      "invoice_number": {
        "type": "string",
        "description": "L'identificatore univoco della fattura"
      },
      "line_items": {
        "type": "array",
        "items": {
```

```
            "type": "object",
            "properties": {
              "description": { "type": "string" },
              "quantity": { "type": "number" },
              "unit_price": { "type": "number" },
              "total": { "type": "number" }
            },
            "required": ["description", "quantity", "unit_price", "total"]
          }
        },
        "total_amount": { "type": "number" },
        "currency": {
          "type": "string",
          "enum": ["USD", "EUR", "GBP", "other"]
        }
      },
      "required": ["vendor_name", "invoice_number", "line_items", "total_amount",
"currency"]
    }
}
```

Usa `tool_choice: {"type": "tool", "name": "extract_invoice_data"}` per forzare Claude a chiamare questo strumento e garantire un output strutturato, con Sonnet 5, Opus 5 e la famiglia 4.x. **Fable 5.1 / Mythos 5.1 e Opus 5.5 rifiutano l'uso forzato degli strumenti con errore 400** (Dominio 2 §2.3): con questi modelli, preferisci la Soluzione 3 qui sotto oppure `auto` insieme a un'istruzione che nomina lo strumento e a `strict: true`.

Soluzione 2: modalità strict

Aggiungi `"strict": true` alla definizione dello strumento per garantire la conformità allo schema:

```
{
  "name": "extract_invoice_data",
  "strict": true,
  "input_schema": {
    "type": "object",
    "properties": { ... },
    "required": ["vendor_name", "invoice_number", "line_items", "total_amount",
"currency"],
    "additionalProperties": false
  }
}
```

Requisiti della modalità strict:

- È necessario impostare `additionalProperties: false`.
- TUTTE le proprietà devono essere incluse in `required`.

Soluzione 3: output_config (output JSON diretto)

Invece di `tool_use`, puoi richiedere direttamente un output JSON. Il parametro è `output_config.format`. Il precedente `output_format` di primo livello è **deprecato**: non usarlo nel nuovo codice. Non è più necessaria neanche la vecchia intestazione beta `structured-outputs-2025-11-13`:

```
response = client.messages.create(
    model="claude-opus-5-5",
    messages=[{"role": "user", "content": "Estrai i dati da questa fattura:
..."}],
    output_config={
        "format": {
            "type": "json_schema",
            "schema": {
                "type": "object",
                "properties": {
                    "vendor_name": { "type": "string" },
                    "total_amount": { "type": "number" }
                },
                "required": ["vendor_name", "total_amount"],
                "additionalProperties": false
            }
        }
    }
)
```

> **Helper dell'SDK:** `client.messages.parse()` (Python/TypeScript) incapsula questo schema: passa un modello Pydantic o uno schema Zod; il metodo invia `output_config.format`, convalida la risposta rispetto allo schema e restituisce l'oggetto analizzato tramite `response.parsed_output`. Quando usi output strutturati, preferiscilo all'analisi manuale del JSON.

Conformità allo schema e correttezza semantica

Concetto importante per l'esame: l'output strutturato garantisce la **conformità allo schema** (tipi corretti, campi obbligatori presenti, valori enum validi), ma NON garantisce la **correttezza semantica**.

Conformità allo schema: ✅ `"total_amount": 150.00` (tipo corretto, campo presente) Errore semantico: ❌ `"total_amount": 150.00` quando la somma delle voci è 175.00

Serve comunque una logica di convalida per individuare gli errori semantici.

Pattern di progettazione degli schemi

Campi obbligatori e facoltativi:

```
{
  "purchase_order_number": {
    "type": ["string", "null"],
    "description": "Numero dell'ordine di acquisto, se presente in fattura; null
se non trovato"
```

```
    }
  }
```

Usa campi nullable quando i documenti di origine potrebbero non contenere l'informazione. Rendere tutto obbligatorio può indurre Claude a inventare dati.

Enum con valore "other":

```
{
  "document_type": {
    "type": "string",
    "enum": ["invoice", "receipt", "contract", "purchase_order", "other"]
  },
  "document_type_detail": {
    "type": ["string", "null"],
    "description": "Se document_type è 'other', descrivi il tipo di documento"
  }
}
```

Enum con valore "unclear":

```
{
  "payment_status": {
    "type": "string",
    "enum": ["paid", "unpaid", "partial", "unclear"]
  }
}
```

Aggiungere `"unclear"` tra le opzioni impedisce a Claude di indovinare quando l'informazione è ambigua.

Limiti di JSON Schema in modalità strict

La modalità strict supporta un sottoinsieme di JSON Schema:

Supportato	NON supportato
`type` (object, array, string, integer, number, boolean, null)	`minimum`, `maximum`, `multipleOf`
`properties`, `required`	`minLength`, `maxLength`
`enum`, `const`	`pattern` (regex), `patternProperties`
`items` (for arrays)	`oneOf`, `if`/`then`/`else`
`anyOf`, `allOf`, `$ref`/`$defs`	Recursive schemas
`additionalProperties: false` (required on every object)	`additionalProperties` set to anything other than `false`

Supportato	NON supportato
`format` (`date-time`, `time`, `date`, `duration`, `email`, `hostname`, `uri`, `ipv4`, `ipv6`, `uuid`)	Complex array constraints

> Le versioni precedenti degli output strutturati non supportavano `anyOf` / `allOf`; ora sono supportati. Gli SDK Python e TypeScript rimuovono dallo schema inviato i vincoli non supportati e li convalidano invece lato client.

Conseguenza: la convalida degli intervalli numerici, della lunghezza delle stringhe, dei pattern regex e degli schemi condizionali complessi deve essere gestita nel codice di convalida, non nello schema.

4.4 Cicli di convalida e nuovo tentativo

Schema

```
max_retries = 3

for iteration in range(max_retries):
    response = extract_data(document, prompt)
    errors = validate(response)

    if not errors:
        return response  # Operazione riuscita

      # Aggiungi al prompt gli errori specifici per il tentativo successivo
    prompt += f"""
      La tua estrazione precedente presentava questi errori di convalida:
    {json.dumps(errors, indent=2)}

      Correggi questi problemi specifici e riprova.
    """

    raise ExtractionError(f"Operazione non riuscita dopo {max_retries} tentativi:
{errors}")
```

```
### Cosa rileva la convalida

**Convalida dello schema** (rilevabile in modo affidabile):
- Campi obbligatori mancanti.
- Tipi di dati errati.
- Valori enum non validi.
- Date o indirizzi email malformati.

**Convalida semantica** (richiede logica personalizzata):
- La somma delle voci non corrisponde al totale.
- Date collocate nel futuro quando non dovrebbero.
- Entità referenziate inesistenti.
```

```
- Contraddizioni tra campi.

### Schemi di autocorrezione

**Valore calcolato e valore dichiarato:**
```

```
{
  "stated_total": 150.00,
  "calculated_total": 175.00,
  "conflict_detected": true,
  "conflict_note": "Il totale dichiarato (150 $) non corrisponde alla somma delle
voci (175 $)"
}
```

Estraendo sia il valore dichiarato sia quello calcolato, puoi rilevare automaticamente le discrepanze.

Quando i nuovi tentativi non funzionano

I nuovi tentativi sono inefficaci quando l'informazione **è assente dal documento di origine**. Se la fattura non contiene un numero d'ordine di acquisto, riprovare non lo farà comparire.

Soluzione: usa campi nullable/facoltativi e accetta `null` per i dati mancanti, invece di riprovare.

Tracciamento dei falsi positivi

Aggiungi un campo `detected_pattern` per tracciare quali costrutti del codice attivano i risultati:

```
{
  "issue": "Possibile SQL injection",
  "detected_pattern": "concatenazione di stringhe nella query",
  "file": "users.py",
  "line": 42
}
```

In questo modo puoi analizzare gli schemi di esclusione: se gli sviluppatori ignorano sistematicamente i risultati «concatenazione di stringhe nella query», potrebbe essere necessario perfezionare i criteri del prompt per quel pattern.

4.5 Elaborazione batch

API Message Batches

Funzionalità	Dettaglio
Costo	Risparmio del 50% rispetto all'elaborazione sincrona

Funzionalità	Dettaglio
Latenza	Fino a 24 ore, senza SLA
Correlazione	`custom_id` per richiesta: i risultati arrivano in **ordine qualsiasi**, quindi usa `custom_id` come chiave, mai la posizione
Più turni	NON supportati in una singola richiesta batch
Output massimo	Normalmente 128k; fino a **300k** con l'intestazione beta `output-300k-2026-03-24` su Opus 5.5 / 5 / 4.8 / 4.7 / 4.6 e Sonnet 5 / 4.6 (non Fable 5.1)
Stati del risultato	`succeeded`, `errored`, `canceled`, `expired`: interroga `batches.retrieve(id).processing_status` finché non è `"ended"`, poi trasmetti i risultati con `batches.results(id)`

Creazione di un batch

```
batch = client.messages.batches.create(
    requests=[
        {
            "custom_id": "invoice-001",
            "params": {
                "model": "claude-sonnet-5",
                "max_tokens": 4096,
                "messages": [
                  {"role": "user", "content": f"Estrai i dati da:
{invoice_001_text}"}
                ],
                "tools": [extract_tool],
                "tool_choice": {"type": "tool", "name": "extract_invoice_data"}  #
forzato: solo Sonnet 5 / Opus 5 / 4.x; errore 400 su Opus 5.5 / Fable 5.1
            }
        },
        {
            "custom_id": "invoice-002",
            "params": { ... }
        }
    ]
)
```

Quando usare batch o modalità sincrona

Sincrona	Batch
Controlli di revisione del codice prima del merge	Analisi del codice durante la notte
Interazioni utente in tempo reale	Rapporti di audit settimanali
Passaggi CI bloccanti	Generazione notturna di test
Debug interattivo	Elaborazione massiva di documenti

Gestione degli errori

Se alcune richieste falliscono, non inviare nuovamente l'intero batch. Usa `custom_id` per individuare e reinviare solo quelle non riuscite:

```
failed = [r for r in results if r.status == "failed"]
resubmit = [orig for orig in original if orig.custom_id in [f.custom_id for f in
failed]]
```

Perfezionamento prima del batch

Prima di elaborare grandi volumi in batch, perfeziona sempre i prompt su un piccolo campione. In questo modo individui i problemi prima di sprecare crediti su 10.000 documenti.

4.6 Revisione in più passaggi

Il problema dell'autorevisione

Quando Claude genera codice e poi lo revisiona nella stessa sessione, conserva il contesto del ragionamento usato per generarlo. «Ricorda» perché ha preso ogni decisione ed è quindi meno incline a metterla in discussione.

Questo fenomeno è chiamato **bias del contesto di ragionamento**.

La soluzione: sessioni di revisione indipendenti

```
Sessione 1: Claude genera il codice
                    ↓
            (sessione completamente separata)
                    ↓
Sessione 2: Claude revisiona il codice
           (senza conoscere le ragioni delle decisioni prese)
```

Architettura in più passaggi per PR di grandi dimensioni

Passaggio 1: analisi locale per file

- Esamina ogni file in modo indipendente.
- Mantieni lo stesso livello di approfondimento per tutti i file.
- Individua bug, problemi di stile, errori di tipo e vulnerabilità di sicurezza.
- Usa un prompt distinto per ogni file.

Passaggio 2: integrazione tra i file

- Esamina le interazioni tra i file.
- Usa un prompt diverso, incentrato sugli aspetti d'integrazione.

- Individua problemi nel flusso dei dati, incompatibilità tra interfacce e gestione degli errori mancante ai confini.
- Riceve i risultati del Passaggio 1 come contesto.

Perché non usare un solo passaggio?

Per PR di grandi dimensioni (almeno 10 file), la revisione in un unico passaggio presenta:

1. **Dispersione dell'attenzione** — Il modello si concentra sui primi file e dedica meno attenzione ai successivi.
2. **Criteri incoerenti** — La qualità della revisione varia da un file all'altro.
3. **Problemi di integrazione non rilevati** — Esaminando un file alla volta in un unico passaggio non si individuano i problemi che coinvolgono più file.

4.7 Thinking, effort e struttura attuale delle richieste

La struttura delle richieste per Claude è cambiata notevolmente tra le generazioni 4.6 e 5. Diversi parametri considerati standard nei materiali meno recenti ora restituiscono **400** con i modelli attuali.

Il thinking adattivo ha sostituito i budget di thinking

```
response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=16000,
    thinking={"type": "adaptive", "display": "summarized"},
    output_config={"effort": "high"},
    messages=[...],
)
```

Modello	**Configurazione thinking**	**Se si omette** `thinking`	`budget_tokens`	`temperature` / `top_p` / `top_k`
Claude Fable 5.1 / Mythos 5.1	`{"type": "adaptive"}` oppure omettere; `disabled` → 400	Esegue thinking adattivo (sempre attivo)	**400**	**400**
Claude Fable 5	`{"type": "adaptive"}` oppure omettere	Esegue thinking adattivo (sempre attivo)	**400**	**400**
Claude Opus 5.5	`{"type": "adaptive"}` oppure omettere; `disabled` → 400 con qualsiasi effort	Esegue thinking adattivo (sempre attivo); **effort predefinito** `medium`	**400**	**400**
Claude Opus 5	`{"type": "adaptive"}` oppure omettere; `disabled`	Esegue thinking **adattivo** per	**400**	**400**

Modello	Configurazione thinking	Se si omette `thinking`	`budget_tokens`	`temperature` / `top_p` / `top_k`
	accettato solo con effort `high` o inferiore	impostazione predefinita		
Claude Opus 4.8 / 4.7	`{"type": "adaptive"}`	Esegue **senza** thinking	**400**	**400**
Claude Sonnet 5	`{"type": "adaptive"}`	Esegue thinking adattivo	**400**	**400**
Claude Haiku 4.5	`{"type": "enabled", "budget_tokens": N}`	Nessun thinking	Obbligatorio per il thinking	Consentiti

La distinzione da ricordare: con **Opus 4.8/4.7 devi impostare esplicitamente** `adaptive`, altrimenti il thinking non viene eseguito; con **Opus 5 il thinking è attivo per impostazione predefinita**; con **Opus 5.5 non può essere disattivato**. Un'impostazione `disabled` ereditata da 4.8 o Opus 5 causa un errore 400; riduci invece `effort`. È comunque preferibile farlo: con il thinking disattivato, Opus 5 a volte inserisce la chiamata a uno strumento nel testo visibile invece di emettere un blocco `tool_use`, quindi la chiamata non viene mai eseguita e non si verifica alcun errore.

Effort

`effort` è disponibile in versione stabile (senza intestazione beta) e si trova **all'interno di** `output_config`, non al livello principale:

```
output_config={"effort": "xhigh"}   # low | medium | high | xhigh | max
```

L'API usa `high` come valore predefinito per tutti i modelli che supportano effort, **tranne Opus 5.5, che usa** `medium`: un livello sotto Opus 5. Dopo una migrazione, quindi, una richiesta che omette `effort` avrà costi e profondità di ragionamento inferiori; impostalo esplicitamente e ripeti la valutazione (Opus 5.5 ragiona più a lungo per turno di Opus 5 allo stesso livello, soprattutto con `xhigh` / `max`, quindi lascia margine in `max_tokens`). Impostare effort sul valore predefinito del modello equivale a ometterlo. Haiku 4.5 rifiuta `effort` e `xhigh` non è disponibile su Opus 4.6 / Sonnet 4.6. Claude Code configura effort **per modello** (`/effort`): il livello salvato per un modello non viene trasferito a un modello rilasciato successivamente e `maxEffortLevel` lo limita a livello organizzativo. Indicazioni Anthropic per modello: con **Fable 5.1, Opus 5 e Sonnet 5 inizia da** `high` **e valuta diverse opzioni**; passa a `xhigh`/`max` solo se le valutazioni mostrano margine e usa ampiamente `low`/`medium` per percorsi di routine o sensibili alla latenza (su questi modelli un effort inferiore spesso supera `xhigh` della generazione precedente); con **Opus 5.5 inizia da** `medium` **e prova anche livelli superiori e inferiori**; con **Opus 4.8 / 4.7 inizia da** `xhigh` per attività di programmazione e agentiche. Un effort più basso produce meno chiamate agli strumenti, più consolidate, e output più concisi. L'effort ha un impatto maggiore su questi modelli che su qualsiasi generazione precedente: **ricalibralo quando migri**, senza riutilizzare automaticamente il vecchio valore. Modificare `effort` al livello principale a metà conversazione invalida la cache dei messaggi; su Fable 5.1, Mythos 5.1, Opus 5.5 e Opus 5 è possibile cambiare **effort per messaggio**: un messaggio `role: "system"` con `content` vuoto che include

`output_config.effort` (beta `mid-conversation-output-config-2026-07-01`, Claude API e Google Cloud) mantiene intatto il prefisso e, non contenendo testo, può trovarsi in qualsiasi punto di `messages`.

Visualizzazione del thinking

`display: "omitted"` è ora il valore **predefinito** per Fable 5 / 5.1, Opus 5.5, Opus 5, Opus 4.8/4.7 e Sonnet 5: un cambiamento silenzioso rispetto a Opus 4.6 e Sonnet 4.6, dove il valore era `"summarized"`. Il thinking continua a essere eseguito e addebitato allo stesso modo; cambia solo la visibilità. Se trasmetti il ragionamento agli utenti, con il valore predefinito sembrerà che ci sia una lunga pausa prima dell'output: imposta esplicitamente `display: "summarized"`. La catena di pensiero grezza non viene mai esposta da alcun modello. Con Fable 5.x **e Opus 5.5**, il testo scritto da Claude *tra una chiamata a uno strumento e l'altra* viene restituito come blocchi `thinking` di aggiornamento dello stato, anziché come blocchi `text`; con il valore predefinito, quindi, non è visibile. Un'interfaccia che mostrava queste note con Opus 5 resterà silenziosa tra le chiamate agli strumenti, senza errori. `display: "updates"` (beta `thinking-display-updates-2026-08-18`) restituisce le note di avanzamento come testo leggibile, mantenendo nascosto il ragionamento; `"summarized"` restituisce entrambi.

Il prefill dell'assistente non è più supportato

Precompilare l'ultimo turno dell'assistente per forzare un formato di risposta restituisce **400** con Fable 5 / 5.1, Opus 5.5, Opus 5, Sonnet 5 e tutta la famiglia 4.6/4.7/4.8. Usa invece output strutturati (`output_config.format`) o istruzioni nel prompt di sistema. Questa tecnica di prompt engineering, ancora consigliata in molti materiali meno recenti, non è più utilizzabile.

Messaggi system a metà conversazione

Con Claude Opus 5.5, Opus 5, Opus 4.8, Fable 5 / 5.1 e Mythos 5 / 5.1 (non Sonnet 5, senza intestazione beta) puoi aggiungere `{"role": "system", "content": "..."}` all'**array** `messages`, invece di modificare il campo `system` di primo livello. È un canale operativo sicuro rispetto alla prompt injection e, poiché non modifica il prefisso memorizzato nella cache, è anche economico. Vincoli: deve seguire un messaggio `user` (o un messaggio dell'assistente che termina con l'uso di uno strumento lato server), non può essere `messages[0]` e deve essere l'ultimo oppure essere seguito da un turno dell'assistente.

Con Fable 5.1 sono arrivate due estensioni (entrambe beta): i messaggi **limitati al turno** con `clear_at: "next_user_message"` (intestazione `mid-conversation-system-clear-at-2026-08-21`) vengono applicati per un turno e poi restano nella trascrizione come cancellati. Sono adatti a promemoria per il singolo turno, come «raggruppa le chiamate indipendenti agli strumenti», che Fable 5.1 richiede più spesso di Fable 5 nei loop lunghi. L'altra estensione riguarda le **modifiche agli strumenti**, tramite blocchi `tool_addition` / `tool_removal` (intestazione `mid-conversation-tool-changes-2026-07-01`): dal 2026-09-22, `tool_addition` può includere anche la definizione completa (`inline-tools-2026-09-15`), così una modifica allo schema o l'aggiornamento di versione di uno strumento server non richiede più di modificare `tools`. Entrambe le funzionalità sono descritte nel Dominio 2 §2.6. Opus 5.5 le supporta tutte. Non eliminare mai le copie precedenti di questi messaggi: con Fable 5.1 e Opus 5.5 ciò equivale a modificare la cronologia.

Thinking preservato: la cronologia fa ora parte del contratto

Ogni blocco `thinking` emesso da Fable 5.1 o Opus 5.5 registra il modello che l'ha generato *ed è valido solo rispetto all'esatto prefisso di messaggi `system` e `tools` che lo precede*. Ne derivano due regole:

1. **Vincolo del modello.** Fable 5.1 e Mythos 5.1 leggono i blocchi reciproci e quelli di tutti i modelli precedenti: Opus 5.5, Opus 5, Fable 5, Mythos 5, Sonnet e Haiku. Opus 5.5 legge i blocchi di Opus 5 e dei precedenti Opus / Sonnet / Haiku, ma non quelli di Fable o Mythos. Nessun modello precedente legge i blocchi di Fable 5.1 o Opus 5.5. Quindi la sequenza Opus 5 → Opus 5.5 → Fable 5.1 conserva il ragionamento a ogni passaggio; qualsiasi altro cambio (un fallback da Opus 5.5 a Opus 5, o un instradamento da Fable 5.1 a Opus 5.5) elimina silenziosamente i blocchi illeggibili (senza addebito) e il modello di destinazione pianifica di nuovo.
2. **Vincolo del prefisso.** Modificare, riordinare o rimuovere turni precedenti invalida tutti i blocchi `thinking` successivi. Per impostazione predefinita, l'API applica questo controllo a Fable 5.1 e Opus 5.5 per gli account creati dal 2026-08-31 (riprodurre un blocco dopo una modifica di questo tipo causa un errore 400); per gli account precedenti, il controllo viene eseguito solo se impostano `thinking.block_binding.prefix_mismatch_behavior` (`"error"` o `"drop_block"`). Le cronologie in cui si aggiungono solo nuovi messaggi, la modifica del contesto lato server e la compattazione effettuata dall'API sono sicure, perché il controllo confronta la conversazione *così come l'hai inviata*: non è sicuro invece un riepilogo della coda creato autonomamente (Dominio 5 §5.7).

Se Claude Code, claude.ai, Managed Agents o Agent SDK gestiscono la cronologia, la mantengono già conforme. Se il tuo codice costruisce direttamente `messages`, segui il controllo in tre passaggi della guida alla migrazione: acquisisci i corpi di richieste consecutive, confronta `system` / `tools` / il prefisso condiviso di `messages` e correggi ogni modifica che non sia un'aggiunta. Con l'intestazione `thinking-binding-controls-2026-08-01`, ogni risposta include `input_transformations`: le voci `thinking_dropped` indicano i blocchi rimossi dall'API (`prefix_binding_mismatch` o `model_binding_mismatch`); nelle richieste in cui il controllo non è applicato, cioè per gli account precedenti, le voci `thinking_mismatch_allowed` (dalla data 2026-09-14) indicano i blocchi che *avrebbero* causato un errore ma sono comunque arrivati al modello. Registra ora il secondo tipo di voce per individuare modifiche alla cronologia in produzione prima di attivare il controllo; in Claude Code, `/claude-api preserved-thinking-migration` automatizza il ciclo acquisizione-confronto-riproduzione. Le versioni di settembre di Claude Code hanno corretto diversi casi in cui `--continue` / `--resume` reinviavano messaggi precedenti in forma modificata: lo stesso controllo aveva rilevato il problema nell'harness di Anthropic.

4.8 Cache dei prompt

La cache è un problema di *struttura del prompt*, motivo per cui questa sezione si trova qui e non in una guida operativa.

Corrispondenza del prefisso: la cache si basa su un prefisso esatto. Qualsiasi modifica, anche di un solo byte, in qualsiasi punto del prefisso invalida tutto ciò che segue. L'ordine di rendering è **`tools`** → **`system`** → **`messages`**; quindi, se l'elenco degli strumenti cambia ordine tra richieste, vengono invalidati il prompt di sistema e l'intera conversazione successiva.

Regola di progettazione: prima i contenuti stabili (prompt di sistema fisso, elenco degli strumenti ordinato in modo deterministico); dopo l'ultimo punto di interruzione `cache_control`, i contenuti variabili (timestamp, ID per richiesta, domanda effettiva).

Meccanismo	Dettaglio
Punti di interruzione	Massimo 4 per richiesta (`cache_control: {"type": "ephemeral"}`)
Prefisso minimo	Dipende dal modello: 512 token su Fable 5.x / Mythos 5.x / Opus 5.5 / Opus 5; 1.024 su Opus 4.8, Sonnet 5, Sonnet 4.6/4.5; 2.048 su Opus 4.7; 4.096 su Opus 4.6/4.5 e Haiku 4.5. I prefissi più brevi non vengono memorizzati in cache e non viene mostrato alcun avviso.
Prezzi	Le letture dalla cache costano il 10% del prezzo dell'input (2,5% con Fable 5.1 / Mythos 5.1, ovvero $0,25/MTok; 5% con Opus 5.5, ovvero $0,20/MTok); le scritture con durata di 5 minuti costano 1,25×, quelle di 1 ora 2×. Con Opus 5.5, un turno memorizzato in cache costa un ventesimo di uno non memorizzato: il motivo economico più forte finora per mantenere stabile il prefisso.
Verifica	Controlla `usage.cache_read_input_tokens`. Se resta a zero tra richieste ripetute, qualcosa invalida il prefisso. La **diagnostica della cache** è disponibile in versione stabile su Claude API dal 2026-09-23 (senza intestazione beta): invia `diagnostics: {"previous_message_id": null}` al primo turno e poi l'`id` della risposta precedente; `response.diagnostics` indica esattamente dove divergono richieste consecutive. Il campo è ora sempre presente (`null` se non hai attivato la funzionalità) e l'impronta viene memorizzata solo per le richieste che includono l'oggetto `diagnostics`: includilo a ogni turno, altrimenti una richiesta successiva segnalerà `previous_message_not_found`.
Pre-riscaldamento	`max_tokens: 0` prepara una voce di cache senza generare output; usa la stessa configurazione di thinking e `effort` del traffico reale, altrimenti la voce non verrà mai riutilizzata.

Cause silenziose di invalidazione da verificare: `datetime.now()` nel prompt di sistema, JSON serializzato con ordine delle chiavi non deterministico, insieme di strumenti variabile per richiesta, ID sessione o nome utente interpolati nel prefisso, cambio di `speed` (modalità rapida) a metà conversazione, modifica di `effort` al livello principale o della configurazione thinking tra richieste (usa effort per messaggio su Fable 5.1 / Opus 5.5 / Opus 5) e modifica del budget dell'attività (viene inserito nel prompt).

Interazione con la ricerca degli strumenti: gli strumenti differiti sono esclusi dal prefisso del prompt di sistema e quelli individuati vengono aggiunti inline, quindi `defer_loading` *preserva* la cache. Uno strumento con `defer_loading: true` non può avere anche `cache_control`: imposta il punto di interruzione su uno strumento non differito.